Experiment-17

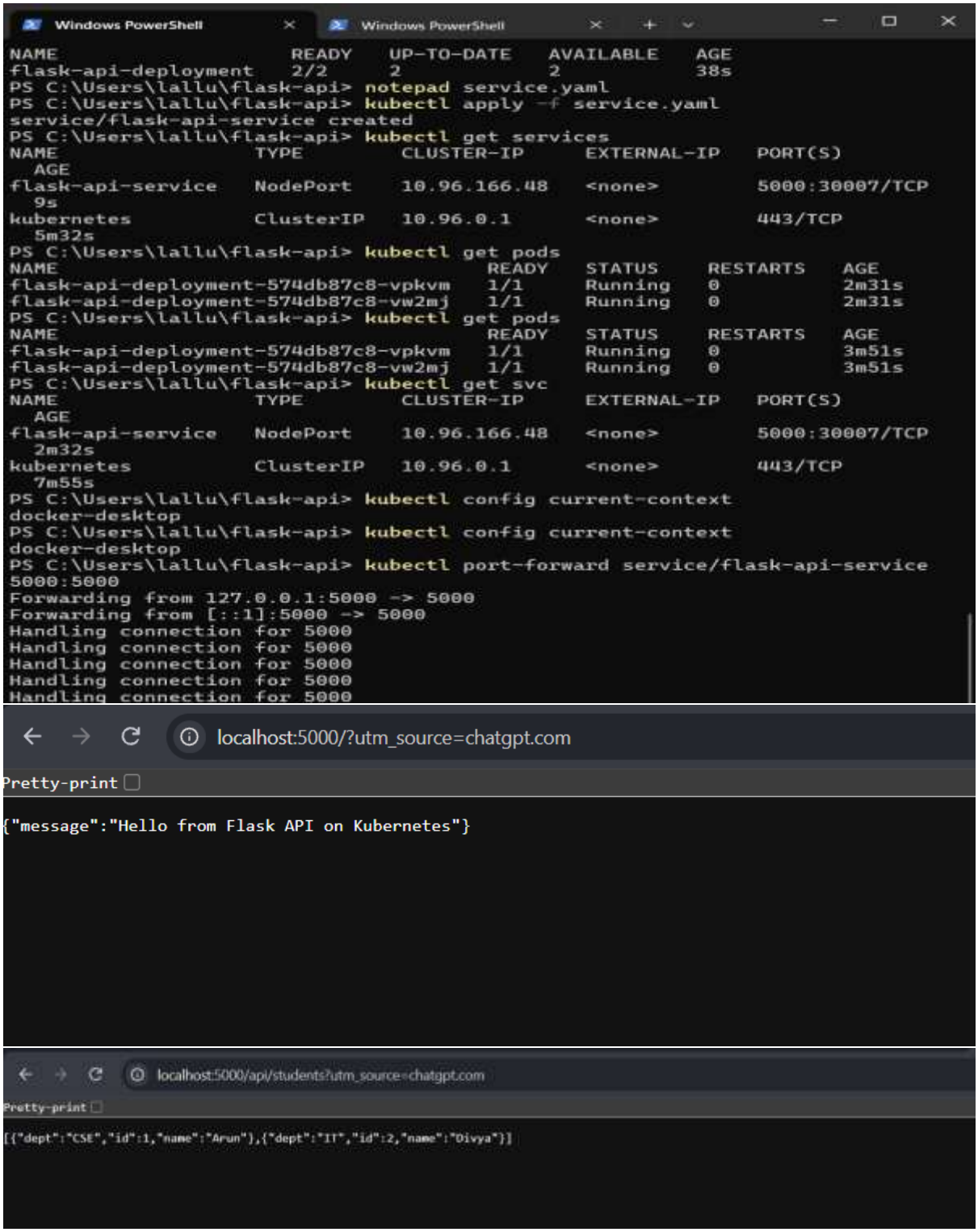
FLASK API CONTAINERIZED WITH DOCKER, PUSHED TO DOCKER HUB AND DEPLOYED ON KUBERNETES

AIM

To create a simple Python Flask API, containerize it with Docker, push the image to Docker Hub and deploy it on Kubernetes using Deployment and Service (Node Port) manifests.

Procedure

1. Created a Flask API application using Python to handle HTTP requests and return responses.
2. Created a requirements.txt file to specify the required Python packages.
3. Created a Docker file to define the application environment and dependencies.
4. Built a Docker image using the Docker build command.
5. Ran the Docker container and tested the Flask API using a web browser.
6. Logged in to Docker Hub using Docker CLI.
7. Tagged the Docker image with the Docker Hub repository name.
8. Pushed the Docker image to Docker Hub using the Docker push command.
9. Created a Kubernetes Deployment YAML file to deploy the Flask API container.
10. Applied the deployment configuration using kubectl apply command.
11. Exposed the application using a Kubernetes Service.
12. Verified the deployment and accessed the Flask API through the service.



Result

Thus, the Flask API was successfully containerized using Docker, pushed to Docker Hub, and deployed on a Kubernetes cluster. The application was tested and verified using Kubernetes services.